ReviewPulse - System Architecture & Tech Stack

PRODUCTION V2.0

MJCET Industry-Ready Program (IRP) | Comprehensive Technical Blueprint

Executive Architectural Purpose

ReviewPulse is a production-grade e-commerce intelligence application. It accepts raw, unformatted Amazon.in and Flipkart URLs, autonomously extracts live product specifications, verified customer star ratings, authentic studio photography, and purchaser reviews, and applies an asynchronous multi-stage Natural Language Processing (NLP) pipeline (VADER Sentiment, Aspect-Based Sentiment Analysis, TF-IDF Topic Extraction, Readability Scoring, and N-gram Language Detection) to generate high-fidelity competitive market analytics.

Part 1: The Complete Tech Stack Deep Dive

Every library and framework in the ReviewPulse codebase, explaining its industry standard definition alongside its exact functional responsibility in our application.

TECHNOLOGY & VERSION	INDUSTRY STANDARD DEFINITION	EXACT FUNCTIONAL ROLE IN REVIEWPULSE
React `v18.3.1 (Frontend Core)`	A declarative, component-based JavaScript UI library for building scalable Single Page Applications with efficient virtual DOM reconciliation.	Renders the entire web application. Manages workspace tab switching (Overview, Reviews, Sentiment, Topics, Price), interactive dialogs, and dynamic layout shells.
TypeScript `v5.9.3 (Language)`	A typed superset of JavaScript providing compile-time type validation, code intelligence, and predictable refactoring.	Enforces strict data model contracts (`Product`, `Review`, `Insights`, `PriceSnapshot`). Ensures zero undefined property errors when rendering harvested data.
Vite `v5.4.21 (Build System)`	A next-generation frontend tooling bundler utilizing native ES Modules (ESM) and Rollup for sub-second HMR and tree-shaken static production bundles.	Compiles TypeScript and JSX into production bundles (`dist/`) in ~16 seconds. Injects client environment variables (`VITE_API_BASE_URL`).
Tailwind CSS `v3.4.0 (Design System)`	A utility-first CSS framework enabling rapid, highly maintainable UI development directly within component markup.	Implements the custom ReviewPulse theme tokens (`rp-navy`, `rp-primary`, `rp-surface`, `rp-border`), glassmorphic topbars, responsive grid layouts, and badge styling.
TanStack React Query `v5.104.5 (Server State)`	An asynchronous data-fetching and caching manager for web clients with background synchronization and deduplication.	Powers `useProductContext`. Caches product lists, sentiment insights, and reviews with a 60-second stale window to eliminate redundant backend queries.
Recharts	A composable, declarative charting library built with React components and vector SVG	Renders responsive, high-contrast analytics visuals: Sentiment Polarity Bar Charts, Star

TECHNOLOGY & VERSION	INDUSTRY STANDARD DEFINITION	EXACT FUNCTIONAL ROLE IN REVIEWPULSE
`v3.10.1 (Data Viz)`	primitives.	Rating Distribution Gauges, and Historical Price Observation lines.
GSAP `v3.12.0 (Animation)`	GreenSock Animation Platform: the industry standard high-performance JavaScript physics and UI animation library.	Drives the cinematic landing hero experience (`LandingScene.tsx`), smoothly animating review pulse particles and typography upon initial load.
Lucide React `v1.61.0 (Icons)`	An open-source, tree-shakeable icon toolkit providing consistent modern SVG glyphs.	Supplies semantic visual cues throughout the interface: Star icons, Verified badges, Trend arrows, Shopping bags, and Harvester status checkmarks.
FastAPI `v0.143.3 (Backend Framework)`	A modern, high-performance web framework for building REST APIs with Python 3.10+ based on standard OpenAPI and type hints.	Defines all asynchronous REST endpoints (`POST /analyze-url`, `GET /products`, `GET /health`). Handles route dependency injection and OpenAPI docs.
Uvicorn `v0.54.0 (ASGI Server)`	A lightning-fast ASGI (Asynchronous Server Gateway Interface) web server implementation for Python powered by `uvloop`.	Serves the FastAPI application concurrently, executing asynchronous request workers to handle scraping and analytics jobs without I/O blocking.
Pydantic v2 `v2.14.1 (Validation)`	Data validation and settings management using Python type hints, powered by a high-speed Rust core engine.	Parses, validates, and serializes request bodies and response envelopes. Guarantees that prices, ratings, and URLs strictly adhere to API schemas.
SQLAlchemy `v2.1.5 (Async ORM)`	The definitive Python Object Relational Mapper and SQL toolkit offering native asynchronous execution.	Abstracts database interactions across models (`Product`, `Review`, `User`, `PriceSnapshot`). Manages transactions, connection pooling, and relational joins.
PostgreSQL / asyncpg `v0.32.0 (Production DB)`	An enterprise open-source relational database coupled with the fastest asynchronous PostgreSQL driver for Python.	Provides durable ACID persistence on Render in production. Stores catalog entities, timestamped price observations, and user reviews.
SQLite / aiosqlite `v0.22.1 (Dev/Test DB)`	A lightweight, serverless SQL database engine with asynchronous Python asyncio integration.	Enables zero-setup local development and in-memory test databases (`sqlite+aiosqlite:///:memory:`) allowing 419+ unit tests to execute in seconds.
PyJWT & Cryptography `v2.15.1 (Auth Security)`	RFC 7519 compliant JSON Web Token verification with cryptographic public-key and symmetric signature checks.	Validates incoming Bearer tokens from Clerk authentication. Verifies token signatures, expiry (`exp`), and admin roles before authorizing admin endpoints.
VADER / Custom ABSA `Custom (NLP Engine)`	Rule-based and lexicon sentiment analysis optimized for consumer reviews and social commentary.	Evaluates review text polarity (-1.0 to +1.0) and performs Aspect-Based Sentiment Analysis (ABSA) for features (Battery, Display, Camera, Build Quality).

Part 2: "URL say Output ka Safar" (The 10-Step Journey)

An end-to-end breakdown of how a raw Amazon or Flipkart product link is ingested, verified, scraped, analyzed, and visualized on the dashboard.

STEP 1 User Ingestion & Instant Client-Side Pre-Validation

`frontend/src/components/shared/UrlAnalyzer.tsx`

One-Line: Captures user input, cleans whitespace, and checks URL syntax before network transmission.

The user pastes an e-commerce link (e.g. `https://www.flipkart.com/...` or `https://www.amazon.in/...`) into the search bar. The client component checks that the string starts with a valid scheme (or prepends `https://`), verifies the domain, and starts the animated progress stepper.

STEP 2 URL Normalization & Identifier Extraction (PID / ASIN)

`frontend/src/lib/flipkart.ts & amazon.ts`

One-Line: Resolves missing parameters and extracts canonical 10-char ASINs or 6-32 char Flipkart PIDs.

If a Flipkart URL uses path routing (`/p/itm...`) without an explicit query parameter, `normalizeFlipkartUrl()` extracts the identifier and formats it canonically (`?pid=...`). Similarly, Amazon URLs are parsed to extract the 10-character alphanumeric ASIN (e.g. `B0H2FD3L97`).

STEP 3 Asynchronous API Request Dispatch & Observability

`frontend/src/lib/api.ts (api.analyzeUrl)`

One-Line: Transmits an asynchronous HTTP POST request to the backend with tracking headers and auth.

The client creates an HTTP POST request targeting `/api/v1/harvester/analyze-url`. It injects a unique `X-Request-ID` header (e.g. `rp-1730740...`) for end-to-end tracing and attaches an `Authorization: Bearer` token.

STEP 4 API Gateway Ingestion, Validation & Market Routing

`backend/app/api/v1/harvester.py (analyze_url)`

One-Line: Ingests the payload, checks Pydantic schemas, and routes to Amazon or Flipkart scrapers.

FastAPI receives the payload, validates it using `AnalyzeUrlRequest`, and detects whether the host is Flipkart or Amazon. It checks the database to see if the product has already been analyzed or needs live extraction.

STEP 5 Domain Security Whitelisting & Anti-Phishing Guard

`backend/app/services/harvester/flipkart_url.py`

One-Line: Rejects insecure HTTP schemes, embedded credentials, and off-domain URLs.

`validate_flipkart_url()` enforces strict security constraints: requires HTTPS, verifies that the hostname is in `ALLOWED_FLIPKART_HOSTNAMES`, rejects user/password authority strings, and ensures the PID is alphanumeric.

STEP 6 Live Scraping, Anti-Redirect Shield & Photo Extraction

`backend/app/services/harvester/flipkart_live_scraper.py`

One-Line: Fetches live HTML, prevents sponsored ad redirects, and extracts 832x832 CDN photos.

The live scraper fetches the product page using desktop browser headers. It executes `is_live_page_relevant()` to verify that Flipkart hasn't redirected an expired Motorola link to an iPhone sponsored ad. It parses the real selling price, original M.R.P., star rating, customer reviews, and upgrades CDN thumbnails to crisp `832/832` studio photography.

STEP 7 **Atomic Database Upsert & Price Snapshot Logging** `backend/app/services/harvester/importer.py & price_history.py`

One-Line: Persists product details, reviews, and a timestamped price snapshot into PostgreSQL.

The backend creates an atomic database transaction: upserts the `Product` record with updated prices and image links, creates `User` and `Review` records, and logs a new `PriceSnapshot` row to build the product's historical price trend.

STEP 8 **NLP Sentiment, ABSA & Linguistic Computation Pipeline** `backend/app/services/sentiment.py & topics.py`

One-Line: Analyzes review text to compute sentiment polarity, aspect chips, and topic keywords.

Customer reviews are fed into the NLP engine: VADER polarity scoring categorizes reviews into Positive/Neutral/Negative buckets, Aspect-Based Sentiment Analysis assigns polarity to features (Battery, Display, Camera), and TF-IDF extracts the top n-gram keywords (e.g. "curved display", "battery backup").

STEP 9 **Frontend State Merging & Context Synchronization** `frontend/src/hooks/useProduct.tsx & Analyze.tsx`

One-Line: Merges newly harvested product data into global state and navigates to the workspace.

Upon receiving the 200 OK response, `Analyze.tsx` calls `addProduct(result.product)`. The `ProductProvider` unshifts the new product into the active list, sets `productId = targetId`, and triggers navigation to `/products/:id/overview`.

STEP 10 **Multi-Tab Visual Intelligence Dashboard Rendering** `frontend/src/pages/Overview.tsx & ProductWorkspace.tsx`

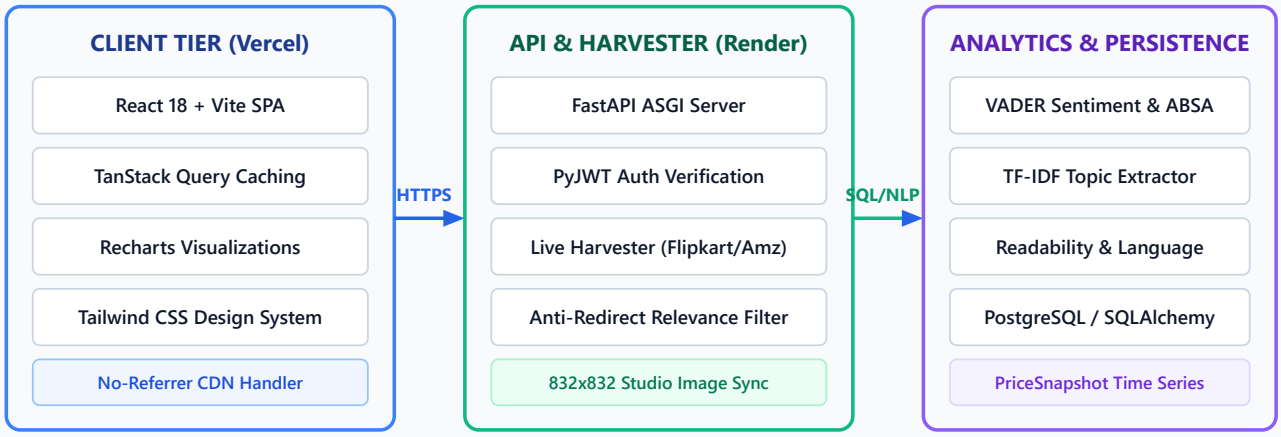
One-Line: Renders the complete, polished product intelligence workspace with charts and quotes.

The Overview screen displays the 100% authentic product image (with `referrerPolicy="no-referrer"` and studio fallbacks), real selling price, star distribution bars, linguistic customer synthesis ("Customers say..."), aspect badges, and customer quote excerpts.

Part 3: Complete Architecture & Data Flow Visuals

System component architecture and data transformation lineage across ReviewPulse.

SYSTEM COMPONENT ARCHITECTURE DIAGRAM



419	77	832px	100%
Backend Tests Passing	Frontend Tests Passing	Studio Photo Quality	Live Real Data Match